
Module 11 — Messaging with Kafka

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — An Idempotent Stock Updater (Easy)	2
Exercise 2 — Poison Messages to the Dead Letter Topic (Medium)	3
Exercise 3 — The Outbox Relay (Hard)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/11-messaging-kafka/exercise/`. Find the `TODO` comments.
2. The tests run against a real Kafka broker and PostgreSQL (Testcontainers). **Docker must be running.**
3. Run the tests:

```
./mvnw -Pexercises -pl modules/11-messaging-kafka/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/11-messaging-kafka/solution/`.

★ Tip

Every exercise has its own topics (`ex1-orders`, `ex2-orders`, `ex3-orders`), and every test uses new random ids. The exercises do not influence each other.

Exercise 1 — An Idempotent Stock Updater (Easy)

Goal: a consumer lowers the stock for every `OrderPlaced` event. Kafka delivers at least once, so the same event may arrive twice. It must still count only once.

Inventory is given: it keeps the stock per book and a set of processed order ids.

Tasks (`exercisel1.StockUpdater`):

- `TODO 1a` — listen to `TOPIC` as consumer group `stock`.
- `TODO 1b` — apply every order only once. `inventory.markProcessed(orderId)` returns `false` if the order was already seen.
- `TODO 1c` — lower the stock of the ordered book.

Hints:

- Lesson section 3.2: `@KafkaListener(topics = ..., groupId = ...)`.
- Mark the order as processed **before** you change the stock, and stop if it was already processed.
- In a real application, the processed ids are stored in the database, in the same transaction as the stock change.

Acceptance criteria: both tests in `ExerciselTest` pass.

Estimated time: 20 minutes

Exercise 2 — Poison Messages to the Dead Letter Topic (Medium)

Goal: an order with a quantity of zero or less can never be paid, and a message that is not JSON can never be read. Both must end up in `ex2-orders.DLT`, and the messages behind them must still be processed.

`PaymentListener` and `DeadLetterTemplates` are given. The settings already use an `ErrorHandlingDeserializer`.

Tasks (`exercise2.ErrorHandlingConfiguration`):

- TODO 2a — a `DefaultErrorHandler` bean. Boot adds it to every `@KafkaListener`.
- TODO 2b — its `recoverer` publishes failed records to `<original topic>.DLT`. Use `DeadLetterTemplates.create(...)`.
- TODO 2c — retry twice, 100 ms apart.
- TODO 2d — an `InvalidQuantityException` is never retried.

Hints:

- Lesson section 3.3 shows the whole configuration.
- The destination resolver receives the failed record: `(record, exception) -> new TopicPartition(record.topic() + ".DLT", -1)`.
- `new FixedBackOff(100, 2)` and `handler.addNotRetryableExceptions(...)`.

Acceptance criteria: all 3 tests in `Exercise2Test` pass. Without your handler, Boot's default handler retries 9 times and then only logs the record, so the DLT tests fail.

Estimated time: 30 minutes

Exercise 3 — The Outbox Relay (Hard)

Goal: `OrderDesk` stores `OrderPlaced` events in the outbox table (given). Write the relay that sends them to Kafka: every event once, in the order it was written, never twice.

Tasks (`exercise3.OutboxRelay.relayBatch`):

- TODO 3a — in one transaction, read at most `batchSize` unsent rows, oldest first, and lock them so that a second relay skips them.
- TODO 3b — send each payload as `OrderPlaced` to `TOPIC`, with `aggregate_id` as the key, and wait for the broker's confirmation.
- TODO 3c — mark each sent row with `sent_at = now()`.
- TODO 3d — return how many rows were sent.

Hints:

- Lesson section 3.6: `SELECT ... WHERE sent_at IS NULL ORDER BY id LIMIT :limit FOR UPDATE SKIP LOCKED`.

- `@Transactional` on the method keeps the lock until the rows are marked.
- `json.readValue(payload, OrderPlaced.class)`, then `kafka.send(...).get(10, TimeUnit.SECONDS)`.
- A record `PendingEvent(long id, String aggregateId, String payload)` lets `JdbcClient` map the rows (`aggregate_id` → `aggregateId`).

Acceptance criteria: all 3 tests in `Exercise3Test` pass.

Estimated time: 45 minutes

Extra Challenge (Optional)

Your relay sends an event and then marks it. What happens if the application crashes exactly between the two steps? Which component of exercise 1 protects the system from the consequence? Then make the relay run every 500 ms with `@Scheduled`, and start two instances of the application: why does `SKIP LOCKED` matter now?